

Orgalyst：类器官明场图像的对话式智能分析系统

一个不依赖大模型也能完整复现的领域工具包，加上一个把它交到实验人员手里的对话式助手。

参赛类别 端到端系统 (End-to-End System)

团队 Orgalyst Lab: Xiaoyan Li (AI / 计算机视觉 / 大模型应用, 项目负责人); 许力文、付旭正 (生物学, 类器官培养与形态学判读, 负责需求与结果审核); 姜翠翠 (AI, 大模型、Agent 架构与编排)

代码 <https://github.com/xiaoyanLi629/orgalyst> (MIT)

权重 <https://huggingface.co/XiaoyanLi/orgalyst-weights>

数据 OrgLine (Zenodo 16355179, CC-BY-4.0), 未使用任何私有数据

1 摘要

类器官与器官芯片实验每天产生大量明场显微照片，实验人员最常问的三件事是“有几个、多大、有没有变化”。现有工具要么是需要调参的脚本，要么是只做分割不管后续统计和留档的软件，把图片变成一份能放进实验记录的报告仍然要花半天。Orgalyst 把这条链路做成两层：下层是一个确定性的领域工具包

orgalyst，包含按器官加载专用权重的 Cellpose 分割、按标定阈值计数的 YOLO 检测、形态测量、质控、组间统计、生长曲线、单文件 HTML 报告和记录输入哈希、权重哈希、参数与软件版本的运行清单；上层是基于 Claude Agent SDK 的对话式助手，通过 MCP 调用这些工具，让实验人员用一句话完成从图片到报告的全过程，而所有数字仍由工具包算出。我们在公开的 OrgLine 数据集（八个来源、五种器官）上完成了五组实验：按器官微调后分割 AP50 在脑、肠、结肠上达到 0.967、0.812、0.815（零样本 cyto3 为 0.005、0.544、0.405）；三器官联合的 YOLO11m 检测 mAP50 0.936 到 0.995，计数阈值在验证集上标定后肺类器官的计数误差从 0.235 降到 0.168；脑类器官 240 张图的自动面积与专家标注的 Spearman 相关 0.963、中位相对误差 1.9%；12 个自然语言任务的 Agent 端到端测试 12 个全部通过，平均每任务 113 秒。我们还如实报告了跨器官泛化的边界、胰腺癌类器官上的不足，以及 Cellpose-SAM 对照实验为什么没有带来收益。

2 问题定义与应用场景

类器官是用干细胞在三维环境中培养出的微型组织，器官芯片则把这类组织放进微流控器件里模拟生理条件。无论哪一种，最便宜、最不伤细胞、频率最高的观察手段都是明场显微镜：每天或每隔几小时拍一张，看它们长得怎么

样。这些照片的分析需求非常固定：数清楚一张图里有多少个类器官，量出每个的大小和形状，比较不同处理组之间有没有差异，把同一个类器官在不同时间点的测量连成生长曲线，最后把结果写成能放进实验记录、能被别人复核的报告。

瓶颈不在算法本身。近几年 Cellpose、OrganoID、OrgaSegment 等工具已经能把类器官分割得相当好，OrgLine 数据集甚至把八个公开数据源整理成了统一格式。真正卡住实验人员的是三件事。第一，工具是给会写代码的人用的：要装环境、选模型、调直径参数，一张图上类器官大小相差十倍时默认参数会失效，而实验人员往往不知道该改哪一个。第二，分割只是第一步，后面的测量、统计、画图、写报告每个实验室都在重复造轮子，而且做法不一致，结果没法横向比较。第三，没有留档：三个月后审稿人问“这个面积是用哪个版本的模型、什么参数算出来的”，多数人答不上来。

Orgalyst 的目标用户是做类器官或器官芯片实验、不写代码的生物学研究者。成功标准是三条：实验人员用自然语言就能拿到一份完整的分析报告；报告里的每个数字都能用命令行在没有大模型的环境下重新算出来，并且和公开数据集自带的专家标注一致；系统对自己不确定的地方要说出来，而不是给一个看起来很准的错误数字。第三条决定了我们把“质控”和“运行清单”做成了工具包的一等公民，而不是附属功能。

3 数据

全部实验使用 **OrgLine** (Zenodo 16355179, CC-BY-4.0)。它把八个已发表的类器官图像数据集统一成了实例分割 (InstanceSeg) 和目标检测 (OrgDet) 两个子集：分割部分有脑 (Schröter 等 2024)、肠 (OrgaSegment)、胰腺导管腺癌 pdac (OrganoID 与 OrganoIDNet) 和结肠 (OrgaExtractor) 四种器官，检测部分有肠 (Tellu、OrgaQuant、OrgaSegment)、脑和肺 (Deep-LUMEN) 三种。我们没有使用任何公司或实验室的私有数据，也没有做额外标注。

原始数据经 `scripts/prepare_data.py` 统一为 8 位 PNG 图像与 uint16 实例掩码（每个类器官一个整数编号），检测标注为 YOLO 文本格式；真彩色明场图保留 RGB，三通道相同的图转为灰度，16 位图按 1 到 99 百分位拉伸到 8 位。分割子集沿用 OrgLine 自带的划分；检测子集按 70 / 15 / 15 随机划分 (seed 0)，划分文件随仓库提供。像素的物理尺寸只有脑数据集给出了明确标定 (Lab A 1024 像素宽的图为 3.1646 $\mu\text{m}/\text{px}$, Lab B 1388 像素宽为 2.9940 $\mu\text{m}/\text{px}$)，其余来源论文只写了仪器和物镜，我们不做估算，形态学结果对这些来源只以像素为单位报告，并在报告里注明。

分割子集 (图像数 / 实例数)	train	val	test
brain	1431 / 1431	186 / 186	240 / 240
intestine	184 / 11977	35 / 2551	12 / 973
pdac	765 / 52281	193 / 12296	20 / 10268
colon	15 / 1487	5 / 0	10 / 750

检测子集（图像数 / 标注框数）	train	val	test
intestine	1969 / 36914	421 / 7631	423 / 8443
brain	1299 / 1299	278 / 278	280 / 280
lung	2804 / 51217	600 / 10903	602 / 10702

脑类器官每张图恰好一个类器官（240 张测试图来自 4 个克隆 × 3 个个体 × 10 个时间点 × 2 个实验室），因此它同时是生长曲线和“与专家标注比面积”实验的天然材料；pdac 的测试图每张有 500 个以上的小实例，训练图平均只有 68 个，这一域内分布偏移是第 7、8 节讨论的主要失败来源；结肠只有 15 张训练图，验证集没有掩码。

4 方法与系统

4.1 总体架构

系统分两层。**工具包层** `orgalyst` 是一个普通的 Python 包，命令行入口 `python -m orgalyst analyze | count | track | report`，不需要任何大模型；它的每一次运行都在一个目录里留下 `manifest.json`（输入文件的 md5、模型标识与权重哈希、全部参数、numpy / torch / cellpose / ultralytics 版本、GPU 型号、逐步日志）、`tables/`（逐实例特征表、汇总、质控、比较、生长表）、`masks/`、`overlays/` 和一份单文件 `report.html`。**助手层**是我们此前为实验室搭建的 BioAgent（Claude Agent SDK + 权限门卫 + 终端与网页两种界面）的精简副本，工具包通过一个 MCP 服务器（`orgalyst/mcp_server.py`，stdio 传输）暴露六个工具：`list_models`、`analyze_images`、`count_organoids`、`compare_groups`、`growth_curves`、`run_summary`（以及 `list_runs`）。助手的系统提示里写明了什么时候用哪个工具、回答时必须区分像素与微米、贴边实例默认排除等约定；助手不看图、不算数，只做需求理解、参数选择和结果解释。

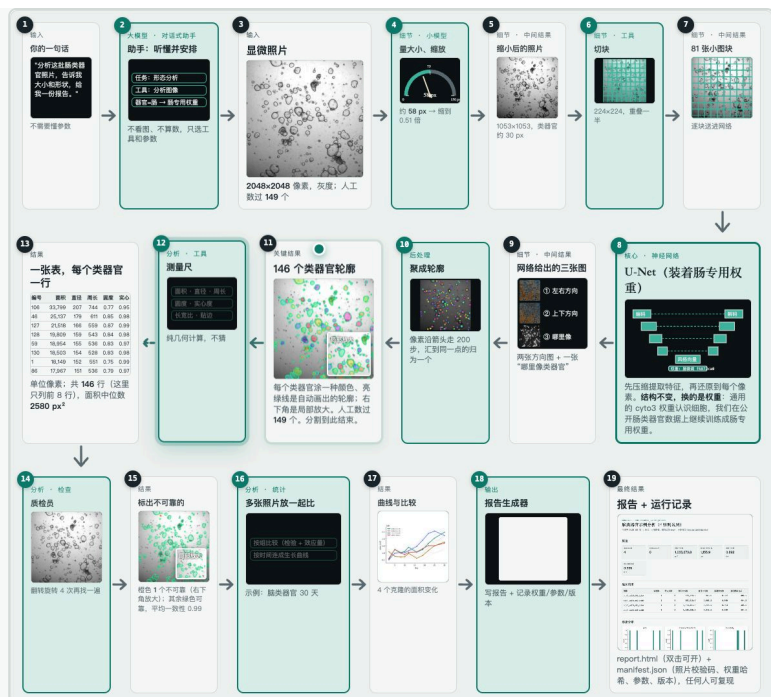


图 1 · 一张肠类器官照片的完整旅程。白卡片是数据，绿盒子是模型或工具，大卡片是关键环节。从“一句话”到助手选工具，再到量大小、缩放、切块、U-Net（装着肠专用权重）、三张输出图、聚成轮廓、146 个类器官轮廓、测量表、质检、统计、报告生成器，最后是报告加运行记录。交互版见 docs/orgalyst_scene.html。

4.2 分割：Cellpose 架构与按器官的专用权重

分割用 Cellpose (Stringer 等, Nature Methods 2021)。它的核心不是某种特定的网络，而是把“实例分割”改写成“回归一个流场”：训练时对每个人工标注的实例做一次以中心为热源的热扩散，得到每个像素指向所属实例中心的方向；网络（一个带残差块和跳跃连接的 U-Net）学习预测这两张方向图和一张“是不是目标”的概率图；推理时把每个前景像素沿方向图迭代移动约 200 步，汇聚到同一点的像素归为同一个实例。这个设计让它天然能处理粘连、非凸和大小悬殊的目标，也让“换骨干网络”成为可能，Cellpose-SAM 就是同一团队 2025 年把骨干换成 ViT 的版本。

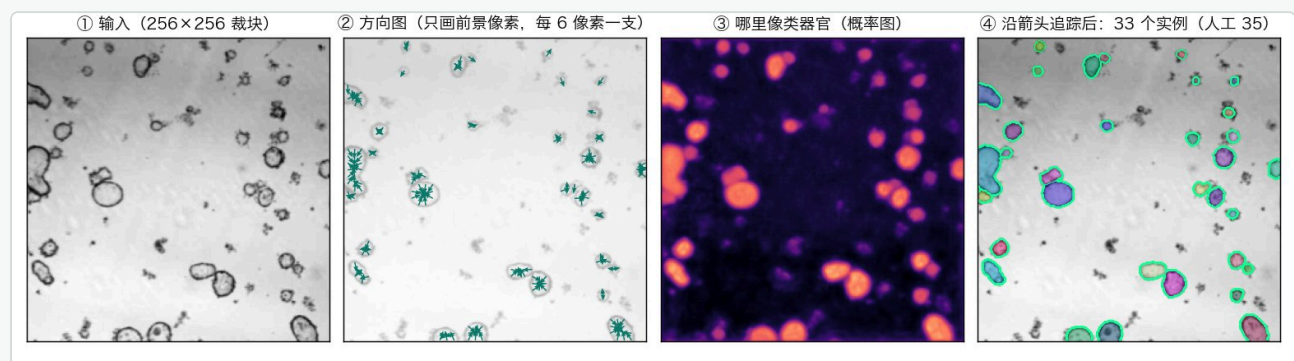


图 2a · 网络实际输出与追踪。肠类器官测试图的一个 256×256 裁块：网络输出两张方向图（这里合成箭头画出）和一张概率图；把概率大于 0 的像素沿箭头走 200 步，落到同一点的归为一个实例，得到 33 个实例，人工标注 35 个。粘连的两个类器官因为箭头指向不同的中心而被分开。

训练时方向图的来源：对一个人工标注的非凸实例（实心底 0.91）以中心为热源反复扩散，热只在实例内传播

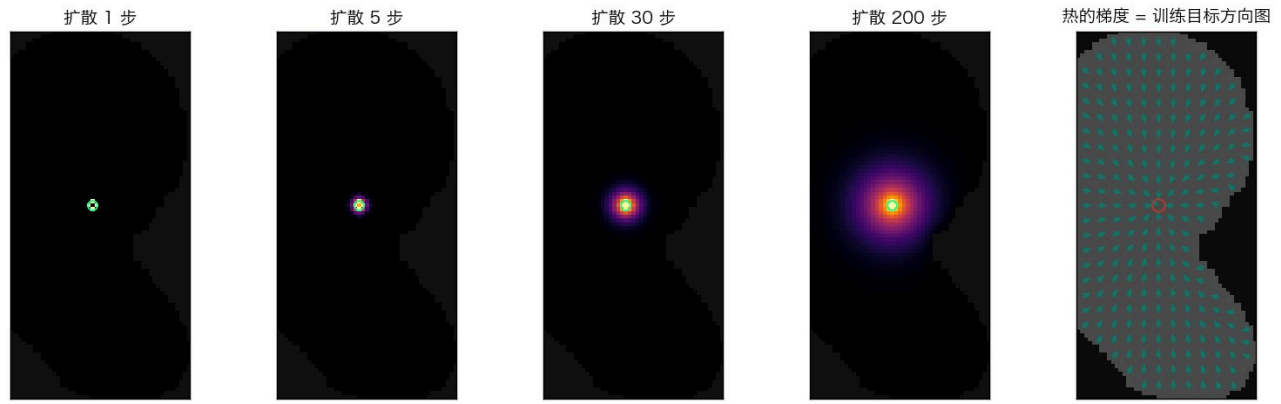
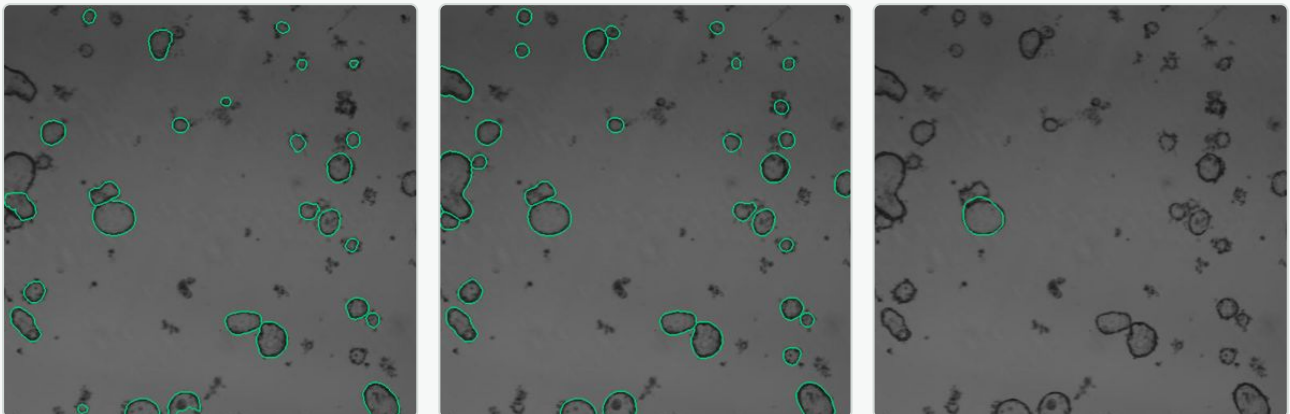


图 2b · 训练目标是怎么来的。训练时不需要人画箭头：对每个人工标注的实例，以其中中心为热源反复做热扩散（热只在实例内部传播，每步取四邻平均），几十步后热场沿着实例形状铺开，取热场的梯度就是每个像素应该指向的方向。这一步只依赖标注掩码，所以任何有实例标注的数据都能直接训练。

Cellpose 有一个容易被忽视的前提：网络是在目标约 30 像素宽的尺度上训练的，推理前要先估计图中目标的典型直径，把图缩放到这个尺度。估错直径的后果很直接：估小了图被放大，一个类器官被拆成几块；估大了图被缩小，所有类器官糊成一团。Cellpose 自带一个从网络风格向量回归直径的尺寸估计器，但它是在细胞照片上训练的，对脑类器官这种 400 像素宽的大目标会给出约 30 像素的荒谬估计，导致零样本时脑的 AP50 只有 0.005。



估成 12 px（偏小）：找到 27 个，
大类器官被拆碎

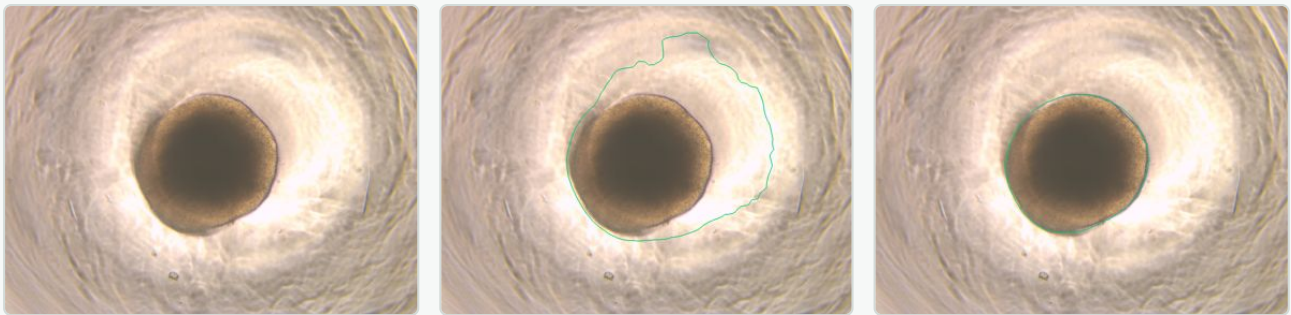
自动估计 29 px：找到 33 个，人工
数过 35 个

估成 150 px（偏大）：只找到 1 个

图 2c · 同一张肠类器官图、同一个模型，只改直径估计值。

我们的做法是按器官加载专用权重，并为每种器官选择一个直径策略。权重由通用的 cyto3 在 OrgLine 各器官训练集上继续训练 200 轮得到（第 6 节），训练时 Cellpose 会把训练集实例的中位直径记进权重

（`diam_labels`）。直径策略有四种，通过 `orgalyst/config.py` 按器官注册：`model` 直接用权重里的训练直径（结肠）；`sizemodel` 用 cyto3 的尺寸估计器逐图估计（肠、pdac，这两种器官图与图之间放大倍率不同）；`refine` 两遍推理，第一遍用训练直径，第二遍用第一遍找到的目标的中位等效直径，钳在 0.25 到 4 倍之间（脑，因为脑类器官从第 2 天到第 30 天面积变化近五倍，单一直径会在两端失效）；`per_image_gt` 用真值直径，只作实验上限。不认识的器官退回零样本 cyto3 加尺寸估计器，并在报告里标注“通用模型，精度较低”。



通用 cyto3 + 自动直径 (估成 30 px): 找到 0 个

通用 cyto3 + 直径改为 404 px: 找到 1 个

脑专用权重 (直径记在模型里): 找到 1 个

图 3 · 脑类器官上，通用模型失败的原因是尺度而不是外观；专用权重把尺度记住了。

4.3 检测计数

当用户只问“有几个”时，分割是浪费的：一张 2048 像素的肠类器官图分割要十几秒，检测不到一秒。我们在 OrgLine 检测子集上训练 YOLO11m (输入 1024, 100 轮)，并比较了三个单器官模型与一个三器官联合模型：联合模型在每种器官上都持平或更好，于是系统只保留这一个检测模型。检测计数有一个常被忽略的参数是置信度阈值：ultralytics 默认 0.25 是为算 mAP 设计的，故意保留很多低置信度的框，直接拿来数数会多报两成。我们在各器官的验证集上从 0.05 到 0.90 扫描阈值，取计数平均相对误差最小者 (肠 0.50、脑 0.45、肺 0.40，未知器官用三器官合并最优的 0.45)，写进配置，测试集只用来报告最终误差。

4.4 形态测量、质控、统计与生长曲线

测量用 scikit-image 的 regionprops：面积、等效直径、周长、圆度 ($4\pi \cdot \text{面积} / \text{周长}^2$)、实心度 (面积 / 凸包面积)、长宽比，以及是否贴着图像边缘；给了像素尺寸就同时输出微米单位的列。贴边实例不完整，汇总统计默认排除并在报告里说明数量。**质控**分两级：图像级计算清晰度 (Laplacian 方差)、光照不均 (局部均值的变异系数) 和饱和像素比例；实例级做翻转与旋转的测试时增强，把原图和 4 个二面体变换后的分割结果两两比较，一个实例在变换后被找回的比例低于 0.5 就标为“低可信”，报告里用橙色标出。这样用户拿到的不只是一个数，还有“哪几个不太可靠”。**统计**做组间比较：两组用 Mann-Whitney U，多组用 Kruskal-Wallis，效应量报告 Cliff's delta，多指标用 Holm 校正，输出箱线图和一张比较表。**生长曲线**按元数据 (个体、时间点、组) 把逐图的中位面积连起来，同时给出每个个体相对第一个时间点的倍数和按组的均值曲线。

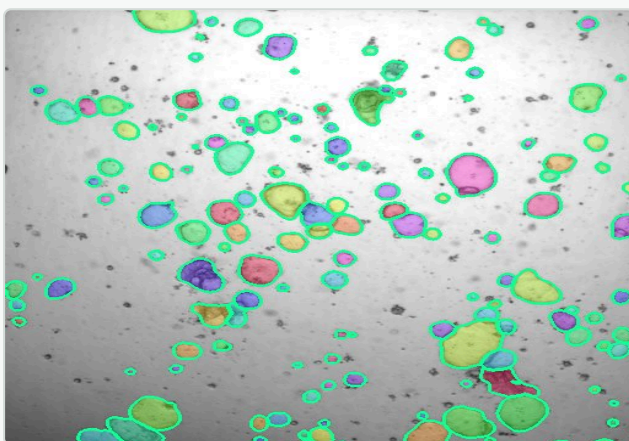


图 4a · 分割结果。肠类器官 2048×2048 测试图，146 个实例，每个涂一种颜色，亮绿线为轮廓；人工标注 149 个。

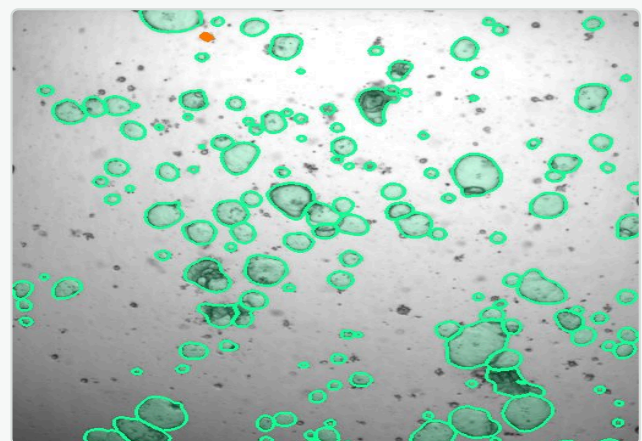


图 4b · 质控。同一张图做 4 次翻转旋转后再分割，橙色为一致性低于 0.5 的实例 (1 个)，其余为绿色；平均一致性 0.99。

4.5 Agent 编排与留档

助手收到请求后先用任务清单工具把工作拆成三到八步并告诉用户打算怎么做，然后逐步调用工具。它需要做的判断其实很少：这批图是什么器官（用户没说就用 generic 并提醒）、有没有像素尺寸（没有就只报像素，绝不编造）、用户要的是“数数”还是“量大小”、要不要质控、分组和时间点信息在哪个文件里。每次工具调用都落在当前对话的产物目录里，回答时引用 run_dir 和报告路径。权限门卫（agent/bioagent/permissions.py）保证原始数据目录只读、其他账号的目录和密钥目录禁止访问，用户可以为每个对话选择确认方式（逐条确认、只确认危险命令、全部自动、只读）。助手还带有按账号隔离的长期记忆（记住用户在做什么项目、常用的数据路径、像素标定和偏好，会话结束时自动归纳），网页版支持多账号、按账号的用量上限和管理员面板；E5 评测时关闭了记忆，以保证各任务相互独立。

开源模型路径。Claude Agent SDK 支持通过 ANTHROPIC_BASE_URL 指向任何兼容 Anthropic Messages API 的网关，我们在 agent/README.md 里给出了 LiteLLM 前置 vLLM 的配置方式，E5 的任务集脚本可以对不同后端各跑一遍。受时间所限，本报告只给出 Claude 后端的结果，这一点在第 8 节作为局限如实列出。

4.6 使用流程与产物

一次典型的使用是这样的。实验人员在网页版或终端里说“分析 /data/exp3 下的 4 张肠类器官照片，做形态分析并生成报告”。助手先列出计划（确认器官与像素尺寸、调用分析工具、读汇总、写回答），然后调用 `analyze_images(images="/data/exp3", organ="intestine", name="exp3")`；工具包在几十秒内完成分割、测量与报告生成，把 run_dir 和汇总返回给助手；助手把总数、逐图数量、面积与圆度的中位数和四分位范围整理成表，并给出报告路径。E5 任务集里助手对这一任务的回答结尾如下（原文节选），它主动说明了单位、指出了没有编造像素尺寸、提示用户先看叠加图核对分割质量，并且明确“201 这个数是模型输出、没有精度保证”，这正是我们希望助手具备的分寸：

“注意：所有尺寸都是像素单位，不是微米。你没有提供 pixel_size_um，我也没有编造标定值。……建议你先翻一下 overlays/ 里的 4 张叠加图。这次用的是肠专用模型，但在没有人工标注做金标准的前提下，201 这个数仍然只是模型输出、没有精度保证。”



图 4c · 工具包生成的单文件报告 report.html（E5 任务 2 的肠类器官批次，4 张图 201 个实例）。依次为概览瓦片（数量、面积 / 直径 / 圆度中位数）、逐图表、分布直方图、叠加缩略图、方法学模板段落（模型、直径策略、排除规则）与溯源表（输入哈希、权重哈希、版本）。报告不依赖网络，双击即可打开，可直接附进实验记录。

5 作为一个分析助手：能问什么，怎么做

把前面的工具包和助手放在一起看，Orgalyst 对使用者呈现的是一个会做类器官图像分析的助手：实验人员用中文或英文描述要做的事，助手按标准流程调用工具、核对结果、用固定格式汇报，并把报告和数据留在本次对话的产物目录里。它不是聊天机器人加几个函数，而是把一套分析规程装进了助手，下面从“能问什么”、“按什么流程做”、“界面长什么样”三个方面说明。

5.1 能问什么

用户会怎么问	助手做什么	用到的工具	用户得到什么
“这批图里有几个类器官？”	用检测模型逐图计数，阈值按器官取标定值	<code>count_organoids</code>	逐图数量表、画了框的图、误差范围说明
“分析这批肠类器官照片，告诉我大小和形状”	按器官加载专用权重分割，测量每个实例，生成报告	<code>analyze_images</code>	总数、逐图表、面积 / 直径 / 圆度的中位数与四分位范围、 <code>report.html</code> 、 <code>features.csv</code>
“像素尺寸是 3.16 μm ，面积换成平方微米”	把标定传给工具，所有长度与面积以微米报告	<code>analyze_images(pixel_size_um=...)</code>	微米单位的结果；没给标定时明确说“只报像素”，绝不估算
“哪些分割结果靠不住？图片质量有问题吗？”	开启质控：图像级指标 + 翻转旋转一致性	<code>analyze_images(qc=True)</code>	清晰度 / 光照 / 饱和指标，橙色标出的低可信实例及其数量
“对照组和处理组的面积有没有差异？”	按分组文件做非参数检验与效应量	<code>compare_groups</code>	各组中位数、p 值、Cliff's delta、箱线图、 <code>compare.html</code> ，结论说人话
“从第 2 天到第 30 天长了多少？”	按个体与时间点连成生长曲线	<code>growth_curves</code>	每个个体首末时间点的数值与倍数、按组的均值曲线、 <code>growth.html</code>
“这批图有肠也有脑，分别分析”	拆成两批，各用对应器官的权重	两次 <code>analyze_images</code>	两个运行目录，各自的数量与形态汇总
“圆度是怎么算的？为什么排除贴边的？”	解释指标定义与规则，不重新计算	(无)	两三句话的解释
“这次用了什么模型、参数？以后怎么复现？”	读运行清单	<code>run_summary</code> / <code>list_runs</code>	<code>manifest.json</code> 的路径及其记录的输入哈希、权重哈希、参数、版本

“分析 /data/x 里的图”（目录是空的 / 器官没说）

说明没有找到图片；或用通用模型并声明精度较低

一句明确的说明，而不是一个看起来正常的数字

表 5-1 · 助手覆盖的问题类型。前十行正是 E5 任务集的十二个任务所覆盖的范围（7.5 节）。

5.2 按什么流程做

光有工具还不够，助手在什么情况下该问什么、先跑什么、怎么核对、怎么汇报，我们写成了一个可复用的流程说明 agent/skillpack/skills/organoid-analysis/SKILL.md，作为 Claude Agent SDK 的本地 skill 自动加载，用户提到类器官图片分析时即生效，也可以用 /organoid-analysis 显式触发。它规定了四步：先确认三个前提（器官类型、像素尺寸、用户要的是数量还是形态还是比较，缺哪个问哪个，但不为简单任务阻塞）；按前提选工具（只数数走检测，其余走分割，图片少或用户关心可靠性时开质控，有分组或时间点信息时接统计）；拿到返回先核对（逐图计数出现 0 或离群值就去看叠加图，贴边排除数和低可信数必须写进回答）；最后按固定的六段格式汇报（一句话结论、逐图表与中位数加四分位范围、单位与排除说明、统计结论、产物路径、至多两条下一步建议）。skill 里同时列出了禁止事项：没有像素尺寸时不给任何微米数字，不用 Python 重算工具已经给出的统计量，不重复跑同一批图。这样助手的行为不再依赖每次提示词写得好不好，换一个模型后端也能沿用同一套流程。

这套流程的效果可以从两处看出来。一是助手的回答有固定的骨架：先说前提（器官、单位），再给结论和表，最后给路径和建议，7.5 节引用的回答就是这样的例子；二是它会主动说“不”：没有像素尺寸时不给微米数，通用模型的结果会注明精度较低，遇到离群的逐图计数会去看叠加图而不是照抄。这些行为写在 skill 和系统提示里，不依赖使用者会不会提问。

5.3 界面



图 5-1 · 网页版界面上的一次真实分析（bio01 账号，4 张肠类器官照片）。中间是助手的回答：前提说明、逐图计数表、整体形态、产物路径与建议；右上角是本对话的确认方式（这里选了“自动允许”），右侧是助手自己维护的任务清单和本次对话的产物文件；底部状态栏显示上下文占用、今日与本月费用、已加载的工具模块。同一套后端也有终端版，供脚本化调用与评测（E5 就是这样跑的）。



图 5-3 · 本次对话的产物文件。右侧面板列出这次对话生成的运行目录、表格和图，可在线预览、下载或打包；“我的数据”页签则是账号自己的上传区。



图 5-5 · 设置页。模型选择、新对话的默认确认方式、按账号隔离的长期记忆（助手在会话结束时自动归纳，用户可编辑或清空）和改密码。

硬件为一台 AutoDL 云主机 (NVIDIA RTX 5090 D, 32 GB 显存; 容器内存上限 62 GB), 软件为 Python 3.10、torch 2.14.0+cu130、cellpose 3.1.1.3、ultralalytics 8.4.152、numpy 2.0.2、scikit-image 0.25。分割微调: cyto3 初始化, 按器官各训练 200 轮, 学习率 0.1 (Cellpose 默认 SGD), 批大小 8, 耗时脑 74 分钟、pdac 31 分钟、肠 22 分钟、结肠 1.4 分钟; 跨器官实验每器官最多取 500 张训练图 (四器官全量 2395 张会超出容器内存被静默杀掉, 这是我们踩过的坑之一)。检测: yolo11m.pt 初始化, 输入 1024, 批 8, 100 轮, 单器官 32 到 65 分钟, 联合模型 126 分钟。Cellpose-SAM 对照在独立的 cellpose 4 环境里跑, batch 2、学习率 1e-5、100 轮, pdac 96 分钟、肠 25 分钟。各阶段的实测耗时见表 6-1。

阶段（单张图，中位数）	512px（51 个实例）	1024px（67 个实例）	2048px（55 个实例）
分割（专用权重 + 逐图尺寸估计）	0.29 s	0.71 s	2.29 s
质控：翻转旋转一致性（4 次）	1.06 s	2.94 s	9.07 s
质控：图像级指标	0.05 s	0.40 s	3.14 s
形态测量 + 汇总	0.03 s	0.06 s	0.11 s
叠加图	0.01 s	0.02 s	0.09 s
检测计数（YOLO11m）	0.01 s	0.01 s	0.01 s
显存峰值	0.7 GB	0.7 GB	0.7 GB

表 6-1 · 各阶段耗时（scripts/bench_timing.py, results/timing.json；RTX 5090 D，肠专用权重 + sizemodel）。只用 CPU 时，512px 图的分割耗时 14 秒，仍可用于小批量。报告生成（含直方图与缩略图）另需约 2 到 5 秒。

MCP 接口以 FastMCP 实现，工具签名如下（全部参数都有默认值，返回 JSON）：`analyze_images(images, organ="generic", pixel_size_um=None, name, diam_mode=None, limit=0, make_report=True, qc=False, meta_csv=None)`；`count_organoids(images, organ="generic", name, conf=None, limit=0)`；`compare_groups(run_dir, groups, metrics=None, exclude_border=True)`；`growth_curves(run_dir, meta_csv=None, pattern=None, metric=None)`；`run_summary(run_dir)`；`list_models()`；`list_runs(limit=20)`。MCP 服务器启动时把 stdout 改道到 stderr，只留一个私有句柄给协议流，避免底层库的打印污染协议，这是我们在接入 Biomni 工具库时总结出的做法。

7 实验与结果

7.1 E1 分割：零样本、按器官微调、直径策略与跨器官泛化

评价指标为实例级 AP50（预测实例与真值实例 IoU ≥ 0.5 记为匹配，用 `cellpose.metrics.average_precision`），同时报告 AP75 与 AP90 反映边界精度。表 1 是主要结果。

器官（test 图数）	cyto3 零样本	按器官微调（model 直径）	微调 + 逐图策略（系统默认）
brain (240)	0.005	0.925	0.967 (refine)
intestine (12)	0.544	0.773	0.812 (sizemodel)
colon (10)	0.405	0.815	0.814 (sizemodel) / 0.800
pdac (20)	0.419	0.564	0.591 (sizemodel)

表 1 · E1 分割 AP50。零样本 cyto3 用自带尺寸估计器；“上限”是给每张图真值中位直径，实际不可用，只用来区分“尺度没估对”和“模型不认识”。

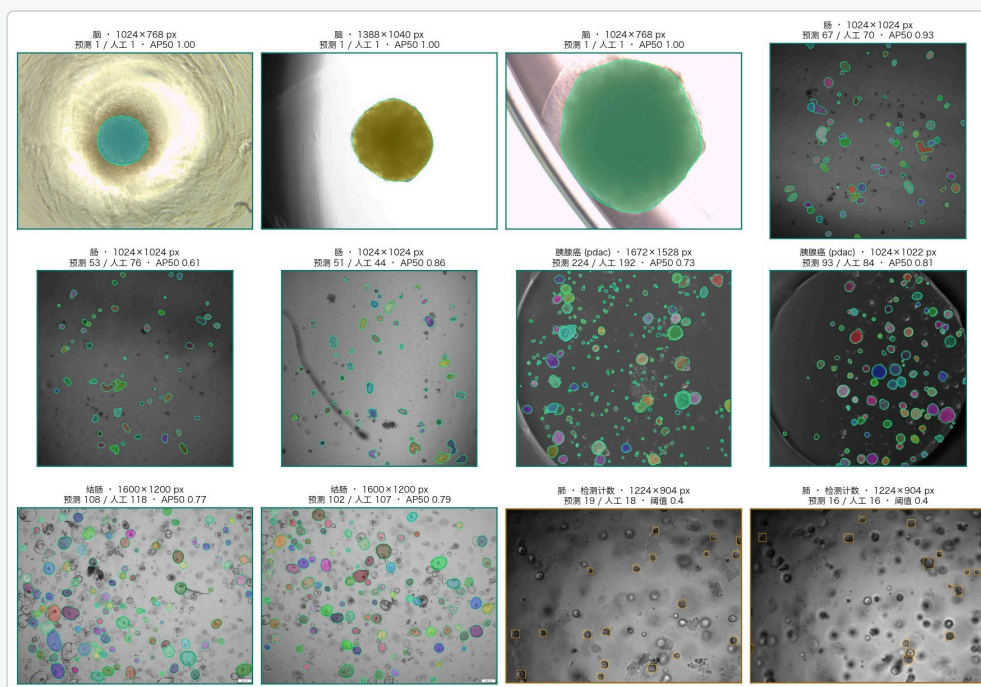


图 7-1 · 系统默认流程在测试集上的示例（12 张，未挑选：每种器官取前几张）。脑三张为不同克隆、不同天数、两个实验室的图，专用权重加两遍推理都准确框出了整个类器官；肠、胰腺癌、结肠为多实例图，彩色填充为预测实例，标题给出预测数、人工数和该图的 AP50；最后两张为肺类器官的检测计数，橙框为检测框。可以看到典型的误差来源：肠第二张的小碎片被漏掉（AP50 0.61），胰腺癌第一张预测多于人工（224 对 192），结肠的贴边与粘连实例。

三点结论。第一，零样本 cyto3 认得类器官（结肠召回 0.985）但精确率只有 0.4 到 0.6，过度分割严重，AP75 只有 0.2 到 0.4，失败主因是尺度估计和边界质量，微调解决了这两点。第二，直径策略的收益是真实的：脑用两遍推理从 0.925 到 0.967，几乎到了真值直径的上限；肠用逐图尺寸估计从 0.773 到 0.812。第三，pdac 仍然弱：测试图每张 500 个以上的小实例，召回只有约 0.5，AP90 接近 0，给真值直径也只到 0.605，说明问题不在尺度而在训练分布（训练图平均 68 个实例，测试图来自另一台仪器）。

跨器官泛化。我们训练了一个四器官联合模型和四个留一器官模型。联合模型在给对尺度时与专用模型持平（真值直径下肠 0.810、pdac 0.606、脑 0.971、结肠 0.789；实际可用的 sizemodel 下肠 0.797、pdac 0.610、结肠 0.789，脑 refine 0.896），但它记住的训练直径 183 像素对任何单一器官都不对，用 model 策略几乎全失效（0.04 到 0.42）。留一器官模型在从未见过的器官上即使给对尺度也只有 0.15 到 0.47（结肠最好 0.465，因与肠形态相近；refine 0.580）。这说明跨器官泛化是有限的，“按器官专用权重”不是设计偏好而是必要的；联合模型只作为未知器官的兜底，它比零样本 cyto3 好。

Cellpose–SAM 对照。Cellpose–SAM（cellpose 4，ViT–L 骨干，宣称不需要直径参数）零样本 AP50 为肠约 0.60、pdac 0.43 到 0.48、脑约 0.03、结肠约 0.37，与 cyto3 零样本同一水平，在脑上同样失败。按器官微调 100 轮后 pdac 0.557（真值直径 0.591）、肠 0.780（0.795），与 cyto3 微调持平而没有超过，权重 1.2 GB（cyto3 微调权重 26 MB），推理慢一倍。我们因此把主模型定为 cyto3 按器官微调，这一对照结果完整保留在 results/ 里。

7.2 E2 检测计数

器官 (test 图数)	联合模型 mAP50	mAP50-95	单器官模型 mAP50	标定阈值	计数 MAPE (
intestine	0.948	0.739	0.946	0.5	0.262 → 0
brain	0.995	0.941	0.995	0.45	0.018 → 0
lung	0.936	0.754	0.931	0.4	0.235 → 0

表 2 · E2 检测。计数 MAPE 为逐图 $|预测数 - 真值数| / 真值数$ 的平均；总数偏差为预测总数 / 真值总数。肠 423 图、脑 280 图、肺 602 图。

联合模型与单器官模型持平或略好，所以系统只保留一个检测模型。阈值标定把肠和肺的计数误差降低了两到三成，把默认阈值下 10% 到 12% 的系统性多报修正为接近 1 的总数偏差；脑类器官一图一目标，任何阈值都接近零误差。剩下的误差主要来自粘连团块和图像边缘的半个类器官，中位相对误差 0.11 到 0.13 说明多数图的计数偏差在一到两个之内。

7.3 E3 与专家标注的一致性

脑类器官原始数据集 (Schröter 等 2024) 除掩码外还给了每张图的人工面积 (像素²)，我们核实它逐张等于 OrgLine 真值掩码的面积，因此可以把它当作专家标注参照。对 240 张测试图跑系统默认流程 (脑专用权重 + refine)，比较自动面积与专家面积。

直径策略	检出 / 漏检	Pearson r	Spearman ρ	中位比值	中位相对误差	≤10% 误差占
model (固定 训练直 径)	230 / 10	0.921	0.959	1.009	2.2%	81%
refine (系统 默认)	237 / 3	0.938	0.963	1.010	1.9%	93%

表 3 · E3 自动面积 vs 专家标注面积，brain test 240 图。

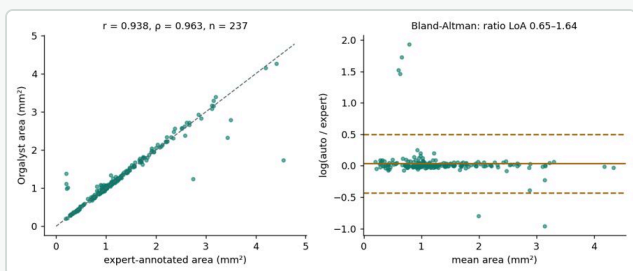


图 5 · 自动面积与专家面积的散点与 Bland–Altman 图 (refine)。残余失败集中在第 2 天最小的类器官（4 张，高估 4 到 7 倍，第一遍推理把培养基背景当成了目标）和一张第 30 天最大的类器官（低估约一半）。

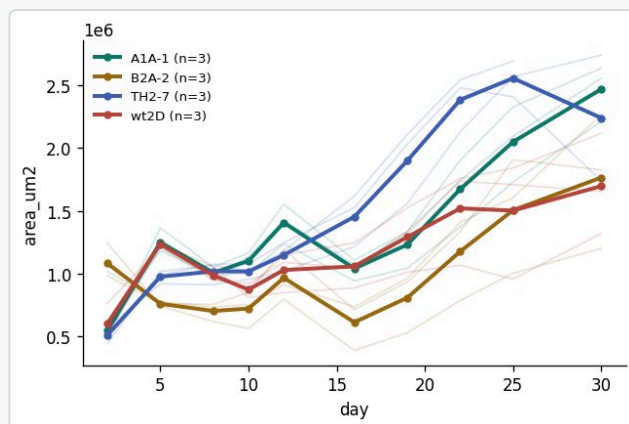


图 6 · 四个克隆的面积生长曲线。由同一批 240 张图的分析结果按文件名解析出个体、克隆与时间点后自动生成，细线为个体，粗线为组均值。

7.4 E4 质控的作用

质控模块的价值在于把错误“标出来”而不是“消掉”。它有没有标对？我们在三种有真值掩码的器官测试集上做了检验：对每个预测实例算它与任一真值实例的最大 IoU，IoU 低于 0.5 记为“错误实例”（多余、拆碎或严重偏移），再看翻转旋转一致性低于 0.5 的“低可信”标记与错误实例的对应关系。

器官	预测实例	标为低可信	错误实例 (IoU<0.5)	低可信组平均 IoU	其余实例平均 IoU
intestine	923	10	87	0.40	0.78
colon	709	17	56	0.48	0.86
pdac	5857	148	876	0.37	0.69

表 7-3 · E4 质控标记 vs 真实分割质量 (scripts/e4_qc_eval.py, results/e4_qc_eval.json)。精确率 = 被标为低可信的实例里确实错误的比例；召回率 = 错误实例里被标出来的比例； ρ 为一致性与 IoU 的秩相关。

三种器官上，被标为低可信的实例确实明显更差：它们里错误实例的占比是其余实例的 3.5 到 6.8 倍，平均 IoU 也低得多，说明翻转旋转一致性是一个有效的可靠性信号。但召回率不高 (0.07 到 0.12)：多数错误实例在翻转旋转后仍然稳定地被错误分割，尤其是 pdac 上大量的小实例漏检本来就不会以实例形式出现，质控标记无法捕捉“漏掉的”，只能捕捉“不稳定的”。因此报告里把它定位为提示而不是过滤：帮助用户把复核精力放在最可疑的实例上，而不是替用户决定删掉谁。

在肠类器官示例图上，120 个实例的翻转旋转一致性平均 0.98，只有 1 个被标为低可信，它正是图像角落一个被截断的团块；在 E5 的质控任务里，4 张肠图 205 个实例中有 3 个被标出。

7.5 E5 Agent 端到端任务集

我们设计了 12 个自然语言任务，覆盖单图分析、批量分析与报告、带物理尺寸的分析、只数数、组间比较、生长曲线、质控、追问指标含义、追问可复现性、器官未知、空目录和混合器官批次。每个任务的数据是从 OrgLine 测试集抽出的 36 张图，成功标准由脚本自动判定：回答里的数值必须落在命令行路径算出的参考值的 5% 到 10% 之内，约定的产物文件必须存在，关键措辞必须出现（例如解释圆度必须提到周长，空目录必须说明没有找到

到图片)，对追问器官这类任务则接受“反问用户”或“用通用模型并说明”两种行为。运行器直接驱动 Agent SDK，不经过终端，每个任务一个新对话（两个追问任务在其父任务的对话里进行），确认方式设为全部自动。

任务	内容	判定	耗时（s）	工具调用	其中 Orgalyst	费用（\$）
t01	单图计数与大小	通过	44	2	1	0.34
t02	批量分析并出报告	通过	64	3	1	0.18
t08	追问：指标含义	通过	65	5	0	0.41
t03	带物理尺寸的分析	通过	49	2	1	0.13
t10	追问：可复现性	通过	28	2	0	0.26
t04	只数数（检测）	通过	40	1	1	0.08
t05	组间比较	通过	309	31	5	1.14
t06	生长曲线	通过	458	46	3	2.04
t07	质控	通过	154	18	1	0.55
t09	器官未知	通过	62	3	2	0.14
t11	空目录	通过	24	2	0	0.11
t12	混合器官批次	通过	65	11	3	0.25

表 4 · E5 结果，Claude 后端（claude-opus-5）。成功率 12/12，平均每任务 10.5 次工具调用（其中 1.5 次为 Orgalyst 工具，其余是任务清单、读文件、检查目录等），平均 113 秒，合计 5.63 美元。

12 个任务全部通过（这一轮在第 5 节的 skill 加入之前完成；加入 skill 后我们复跑了批量分析、追问指标含义和质控三个任务，同样全部通过，平均耗时从 94 秒降到 72 秒，记录在 `results/e5_agent_tasks_skilltest.json`；完整的 12 任务复跑留待下一版）。值得注意的是工具调用的分布：多数任务只需要一到三次调用，助手拿到工具返回的汇总就直接作答；组间比较和生长曲线两个任务分别用了 31 和 46 次调用、5 到 8 分钟，因为助手在拿到统计结果后自行读取了比较表和生长表逐项核对，又用 Python 重算了一遍中位数。这种“多此一举”提高了可信度但增加了时间和费用，在生产环境里可以通过系统提示约束。器官未知的任务里助手没有反问，而是直接用通用模型跑并在回答里注明精度可能较低，这符合我们的约定。

8 可靠性分析与局限

失败案例。分割的两类典型失败都出现在脑类器官的极端尺度上：第 2 天直径只有约 80 像素的类器官在第一遍推理时会被培养基的纹理淹没，refine 的第二遍反而把背景团块当成目标放大了误差；第 30 天直径超过 600 像素的类器官偶尔被切成两半。pdac 的失败是系统性的：测试图的实例密度是训练图的七倍，来源仪器不同，模型漏掉了一半的小实例；这个问题需要更多同源训练数据或针对小实例的多尺度推理，我们没有在比赛期间解决。检测计数在粘连团块上会把两个数成一个，反过来在气泡和碎片上会多数。

跨器官泛化边界。第 7.1 节的留一实验给出了明确的边界：没有见过的器官，AP50 只有 0.15 到 0.47。因此系统对未注册的器官只提供“通用模型 + 明确的低精度提示”，不承诺精度。用户拿到新器官时应当用几十张标注图微调（结肠只用 15 张训练图就从 0.405 到 0.815）。

大模型相关风险。助手可能选错器官、编造像素尺寸、或者在结果不确定时给出过于肯定的措辞。我们的缓解是结构性的：所有数值来自工具，工具返回里带着单位与排除说明，系统提示禁止在未知像素尺寸时估算，权限门卫限制文件访问；E5 任务集就是对这些约束的回归测试。但 12 个任务不能覆盖真实使用里的提问方式，我们只在 Claude 一个后端上评测，开源模型后端的差距没有量化。此外助手依赖付费 API，评审复现 Agent 路径需要自己的密钥；命令行路径不受此限制，报告里的每个数字都可以由它复现。准备附录 A 的演示时我们还观察到一个反例：在一组第 30 天脑类器官的组间比较里，脑模型漏掉了 TH2-7 组三张图中的两张（正是 E3 里漏检的那几张），助手把这个失败放在回答最前面如实说明，这是我们希望看到的；但它接着自己写了一段阈值分割代码“补救”出这两张图的面积并用于检验，这违反了 skill 里“不自己写分割代码”的约定。助手的诚实和越界在同一个回答里出现，说明只靠提示词约束是不够的，下一版需要在权限门卫里对“执行图像分割类代码”做硬性拦截，或让工具在失败时返回明确的“不可比较”状态。附录里的组间比较演示因此改用第 16 天的照片。

数据局限。只有脑数据集有像素标定，其余器官的形态学结果只能以像素为单位；OrgLine 的测试集在肠（12 张）和结肠（10 张）上很小，AP50 的置信区间很宽；数据集全部来自自己发表的公开数据，没有器官芯片的时序数据，生长曲线实验只在脑类器官的静态培养上验证过。质控标记与真实错误的对应关系只在三种器官的测试集上做了初步评估（7.4 节），样本量有限。

9 影响与未来工作

对实验人员而言，Orgalyst 把“拍完照片到拿到报告”从半天缩短到几分钟，而且报告是可以放进实验记录、三个月后仍能复核的。对方法研究者而言，工具包提供了一个按器官注册权重和直径策略的框架，加一个器官只需要放一份权重和一行配置。下一步有三件事最有价值。第一，器官芯片时间序列：芯片上的类器官位置固定，可以把检测框在时间上关联起来做逐个体追踪，而不是像现在依赖文件名里的元数据。第二，剂量响应：在形态学指标上拟合四参数 logistic 曲线，与已知作用机制对照，这是设计文档里的 E6，本次没有来得及做。第三，与实验自动化对接：把助手作为一个 MCP 客户端接到成像系统的调度接口上，让“拍完自动分析、异常时提醒”闭环起来。此外 pdac 上的小实例问题值得单独攻关，可能的方向是滑窗多尺度推理和用检测框引导分割。

10 复现指南

仓库 README 给出了完整命令，这里列出顺序与预期耗时（RTX 5090 D 上）。环境：`pip install -r requirements.txt`（torch 按 CUDA 版本安装，约 10 分钟）；权重：`bash scripts/download_weights.sh`（约 200 MB，国内加 `--mirror`）。快速验证（不需要数据集）：`python -m orgalyst analyze --organ intestine --images 任意明场图目录 --out runs`，几秒到几十秒。复现全部表格：`download_orgline.sh` 与 `extract_orgline.sh`（约 25 GB，视网速）；`prepare_data.py`（15 分钟）；`run_e1_zeroshot.sh`（20 分钟）；`run_e1_finetune.sh`（2.5 小时）；`run_e1_crossorgan2.sh`（3 小时）；`run_e2_det.sh`（4.5 小时）与 `calibrate_det_conf.py`（15 分钟）；`e3_consistency.py`（10 分钟）；`run_cpsam_ft.sh`（需 `setup_cellpose4.sh` 的独立环境，2.5 小时）；`e5_prepare.sh` 与 `run_agent_tasks.py`（需要 agent/ 与 API 密钥，25 分钟，约 6 美元）。所有脚本输出到 `results/`，仓库里已包含我们跑出的全部结果文件，报告中的每张表都能对应到其中一个文件。Docker 镜像（Dockerfile）覆盖命令行路径。

11 外部资源与许可证清单

资源	用途	许可证 / 来源
OrgLine (Zenodo 16355179)	全部训练与评测数据	CC-BY-4.0；不随仓库分发，脚本从原始来源下载
脑类器官原始数据 (Zenodo 10301912, Schröter 等 2024)	E3 的专家面积与像素标定	CC-BY-4.0
Cellpose 3 (cyto3 权重、size_cyto3)	分割骨干与初始化权重	BSD-3, Stringer & Pachitariu
Cellpose-SAM (cpsam / cpsam_v2)	对照实验	BSD-3
Ultralytics YOLO11m	检测计数	AGPL-3.0 (微调权重同许可)
scikit-image、scipy、pandas、matplotlib	测量、统计、绘图	BSD / MIT
Claude Agent SDK、Claude (claude-opus-5)	助手层与 E5 评测	Anthropic 商业 API；命令行路径不依赖
MCP (FastMCP)	工具接口	MIT
Claude Code	开发过程中的编程助手 (代码、文档与实验脚本的撰写均经作者审核)	Anthropic

本项目代码以 MIT 许可发布；微调权重托管于 Hugging Face (XiaoyanLi/orgalyst-weights)，继承各自基础模型的许可证。

附录 A · 功能演示截图

以下截图均来自网页版的真实运行 (bio01 账号，数据为 E5 任务集的测试图，界面为中文)，每张是助手对该请求的最终回答，对应第 5.1 节表格里的问题类型。截图未经修饰，个别回答较长时只截取了主体部分。

服务器日志分析结果

图片	文件大小
image_10.jpg	14
image_30001.png	19
image_30008.png	16
image_30023.png	11
image_30032.png	21
合计 (含删)	87

方法二(参数)

- 主工: `Onpgetconf conf_orignids /YOLO1m 检测图片`, 不做分割, 速度更快
- 输入参数: `lung`, 测试数据集为 0-40 (按 40 张图片)
- 输入尺寸 `imgsz=[1024, 1024]`, 模型 `yaml=[1, 14, 1817, 1280x224]`

警告! 检测图片, 这是检测图片, 不是分析, 在验证这上样本每张图片需要 20% (中心 13s), 如果这样就要求可写设备, 建议打开 `ovtsays` 看一下有没有再补数据复现。

产出文件

```
run_dir/ /autoflows/data/diagnoset/users/hio8t8t/chats/2026-09-17_125508/outputs/outputs/analysis/runs/lung_conf_20260917_12520917_142331_dse544
```

- 检测图片: `tables/confusion.csv`
- 每个检测图 (含坐标、置信度): `tables/boxes.csv`
- 检测标注图: `overlays/`

下一步建议 (没有写需求)

- 查看 `ovtsays/lay` image_30001 (21 点、删除) 随时补充完成一些需求再返回查看记录。
- 此次分析需要一些数据的清洗和预处理, 近期 `analyze_images(formal-url)`, 但是注意数据物归主仍高频, 结合业务是像样本, 上次和样本不一样, 有特定值 (`imgpixel` 文件一屏) 再直接去处理数据。

图 A-1·只数数 (检测)。5 张肺类器官照片，只问每张有几个。助手调用检测计数工具，逐张列出数量并给出合计，说明用的是肺的标定阈值 0.40，不做分割。

[illegible]

图 A-4 · 质控。4 张肠类器官照片，要求标出不可靠的分割。助手开启质控，报告图像级指标和被标为低可信的实例数。

[illegible]

图 A-7 · 生长曲线。org01 从第 2 天到第 30 天的连续照片，元数据在 meta.csv。助手分析后连成生长曲线，报告首末时间点面积与倍数（4.59 倍），随后认为第 2 天的分割只圈住了致密核心，用自己写的纹理法重新量了一遍并给出另一口径（1.35 倍）。观察本身有价值，但自制测量不应作为数字报告，属于第 8 节讨论的越界行为。

结果：面积中位数 1,068,458 μm² (≈ 1.07 mm²)

3 组数据分别占 1 个类类型，与“每个占一个节点，没有实现节点间连接”。

组别	实例数	面积(μm²)	面积(μm²)	面积
org01_v2d3_c01e1c_LabA	1	1,068,458	106,689	0.999
org02_v2d3_c01e1c_LabA	1	1,268,938	126,689	0.998
org03_v2d3_c01e1c_LabA	1	938,383	90,706	0.998
平均节点	3	1,068,458 (中位)	106,689 (99.9)	

其他信息包括 (面积单位, 距 1.646 μm 像素 体积) :

- 有效置信度中位 **1166 μm** (Q1 1211 / Q3 1218)
- 面积 0.898, solidity 0.998, 周长 1.05 > 三个都是接近圆形, 边缘光滑的圆形。

注意: Organelle analysis, organelles 图像由生物图像数据集 (BrainA004064c6cfd, diam_modelvline, 训练数据 404.5 μm), 像素平均半径的 3.646 μm), 默认从脑组织实例 (未连接节点)。我看了 org2 的 overlay, 发现与类似边缘的节点, 无连接情况。

文件树 (img_dir) /autol1-fs/data/bioagent/users/bio181/chats/2026-49-17_cc08b/output/orgnel1_v2d3_brain_vn_morph_20260917_162432_998580

```

├── report.txt  查看置信度的数据
├── tables/features.csv  查看表格数据
├── overlays/  masks/
└── ...

```

一点提示: 1166 μm 的有效置信度中位 6 级像素类直径大 (常见范围 400-1000 μm), 分割本身没问题, 所以结果造成这个偏差真的纳闷。建议重复使用 1.646 μm 这个判定边缘的阈值组合组合。简单的尝试可以选另一个判定值继续, 或者把 8 级像素直径放大 10 次左右。

图 A-2 · 带物理尺寸的分析。3 张脑类器官照片，用户给了像素尺寸 3.1646 $\mu\text{m}/\text{px}$ 。助手把标定传给工具，面积与直径以微米报告。

[illegible]

图 A-5 · 追问：指标含义。在质控对话里追问圆度怎么算、为什么排除贴边、低可信是什么意思。助手只解释定义与规则，不重新计算。

[illegible]

图 A-8 · 混合器官批次。4 张照片里 2 张脑、2 张肠。助手拆成两批，各用对应器官的权重，分别报告数量与面积中位数。

[illegible]

图 A-3 · 追问：可复现性。在上一个对话里追问用了什么模型、直径策略，manifest.json 在哪。助手给出完整路径并说明其中记录的输入哈希、权重哈希、参数与版本。

4.6 安装配置 wvdsl 与 wvdsl-7 驱动和安装

4.6.1 安装驱动 (wvdsl, wvdsl-7 驱动及 wvdsl-gsmnet)

包	安装大小	安装后大小	安装时间	描述
wvdsl	10.0 kB	10.0 kB	17 s	1.88 s
wvdsl-wvdsl-usb-gsm	1.0 kB	1.0 kB	17 s	0.00 s
wvdsl-wvdsl-usb-gsm	1.0 kB	1.0 kB	17 s	0.00 s
wvdsl-7	10.0 kB	10.0 kB	17 s	0.00 s
wvdsl-7-wvdsl-usb-gsm	1.0 kB	1.0 kB	17 s	0.00 s
wvdsl-7-wvdsl-usb-gsm	1.0 kB	1.0 kB	17 s	0.00 s

安装后 wvdsl 1.6.6-4 wvdsl-7 版本 1.6.6-4 wvdsl-7-wvdsl-usb-gsm 1.6-4 wvdsl-7-wvdsl-usb-gsm 1.6-4
 (安装前 - 安装后) (安装前) (安装后) (安装后) (安装后)

4.6.2 安装配置

- `lsmod` 查看内核模块是否安装成功
- `ifconfig wlan0 up` 启动网卡, 并设置 IP 地址, 子网掩码和网关地址为 192.168.1.0/24
- `cd /usr/sbin` 进入目录, 下面两个命令用于设置 wvdsl 的配置文件, 其中 `wvdsl-wvdsl-usb-gsm` 为 `wvdsl` 的 `wvdsl` 配置文件, `wvdsl-wvdsl-usb-gsm` 为 `wvdsl-7` 的 `wvdsl` 配置文件
- `cp /usr/sbin/wvdsl-wvdsl-usb-gsm /usr/sbin/wvdsl-wvdsl-usb-gsm` 复制配置文件
- `cp /usr/sbin/wvdsl-wvdsl-usb-gsm /usr/sbin/wvdsl-wvdsl-usb-gsm` 复制配置文件

4.6.3 安装配置

4.6.3.1 安装驱动和配置

4.6.3.2 安装驱动和配置

4.6.3.3 安装驱动和配置

4.6.3.4 安装驱动和配置

4.6.3.5 安装驱动和配置

4.6.3.6 安装驱动和配置

4.6.3.7 安装驱动和配置

4.6.3.8 安装驱动和配置

4.6.3.9 安装驱动和配置

4.6.3.10 安装驱动和配置

4.6.3.11 安装驱动和配置

4.6.3.12 安装驱动和配置

4.6.3.13 安装驱动和配置

4.6.3.14 安装驱动和配置

4.6.3.15 安装驱动和配置

4.6.3.16 安装驱动和配置

4.6.3.17 安装驱动和配置

4.6.3.18 安装驱动和配置

4.6.3.19 安装驱动和配置

4.6.3.20 安装驱动和配置

4.6.3.21 安装驱动和配置

4.6.3.22 安装驱动和配置

4.6.3.23 安装驱动和配置

4.6.3.24 安装驱动和配置

4.6.3.25 安装驱动和配置

4.6.3.26 安装驱动和配置

4.6.3.27 安装驱动和配置

4.6.3.28 安装驱动和配置

4.6.3.29 安装驱动和配置

4.6.3.30 安装驱动和配置

4.6.3.31 安装驱动和配置

4.6.3.32 安装驱动和配置

4.6.3.33 安装驱动和配置

4.6.3.34 安装驱动和配置

4.6.3.35 安装驱动和配置

4.6.3.36 安装驱动和配置

4.6.3.37 安装驱动和配置

4.6.3.38 安装驱动和配置

4.6.3.39 安装驱动和配置

4.6.3.40 安装驱动和配置

4.6.3.41 安装驱动和配置

4.6.3.42 安装驱动和配置

4.6.3.43 安装驱动和配置

4.6.3.44 安装驱动和配置

4.6.3.45 安装驱动和配置

4.6.3.46 安装驱动和配置

4.6.3.47 安装驱动和配置

4.6.3.48 安装驱动和配置

4.6.3.49 安装驱动和配置

4.6.3.50 安装驱动和配置

4.6.3.51 安装驱动和配置

4.6.3.52 安装驱动和配置

4.6.3.53 安装驱动和配置

4.6.3.54 安装驱动和配置

4.6.3.55 安装驱动和配置

4.6.3.56 安装驱动和配置

4.6.3.57 安装驱动和配置

4.6.3.58 安装驱动和配置

4.6.3.59 安装驱动和配置

4.6.3.60 安装驱动和配置

4.6.3.61 安装驱动和配置

4.6.3.62 安装驱动和配置

4.6.3.63 安装驱动和配置

4.6.3.64 安装驱动和配置

4.6.3.65 安装驱动和配置

4.6.3.66 安装驱动和配置

4.6.3.67 安装驱动和配置

4.6.3.68 安装驱动和配置

4.6.3.69 安装驱动和配置

4.6.3.70 安装驱动和配置

4.6.3.71 安装驱动和配置

4.6.3.72 安装驱动和配置

4.6.3.73 安装驱动和配置

4.6.3.74 安装驱动和配置

4.6.3.75 安装驱动和配置

4.6.3.76 安装驱动和配置

4.6.3.77 安装驱动和配置

4.6.3.78 安装驱动和配置

4.6.3.79 安装驱动和配置

4.6.3.80 安装驱动和配置

4.6.3.81 安装驱动和配置

4.6.3.82 安装驱动和配置

4.6.3.83 安装驱动和配置

4.6.3.84 安装驱动和配置

4.6.3.85 安装驱动和配置

4.6.3.86 安装驱动和配置

4.6.3.87 安装驱动和配置

4.6.3.88 安装驱动和配置

4.6.3.89 安装驱动和配置

4.6.3.90 安装驱动和配置

4.6.3.91 安装驱动和配置

4.6.3.92 安装驱动和配置

4.6.3.93 安装驱动和配置

4.6.3.94 安装驱动和配置

4.6.3.95 安装驱动和配置

4.6.3.96 安装驱动和配置

4.6.3.97 安装驱动和配置

4.6.3.98 安装驱动和配置

4.6.3.99 安装驱动和配置

4.6.3.100 安装驱动和配置

4.6.3.101 安装驱动和配置

4.6.3.102 安装驱动和配置

4.6.3.103 安装驱动和配置

4.6.3.104 安装驱动和配置

4.6.3.105 安装驱动和配置

4.6.3.106 安装驱动和配置

4.6.3.107 安装驱动和配置

4.6.3.108 安装驱动和配置

4.6.3.109 安装驱动和配置

4.6.3.110 安装驱动和配置

4.6.3.111 安装驱动和配置

4.6.3.112 安装驱动和配置

4.6.3.113 安装驱动和配置

4.6.3.114 安装驱动和配置

4.6.3.115 安装驱动和配置

4.6.3.116 安装驱动和配置

4.6.3.117 安装驱动和配置

4.6.3.118 安装驱动和配置

4.6.3.119 安装驱动和配置

4.6.3.120 安装驱动和配置

4.6.3.121 安装驱动和配置

4.6.3.122 安装驱动和配置

4.6.3.123 安装驱动和配置

4.6.3.124 安装驱动和配置

4.6.3.125 安装驱动和配置

4.6.3.126 安装驱动和配置

4.6.3.127 安装驱动和配置

4.6.3.128 安装驱动和配置

4.6.3.129 安装驱动和配置

4.6.3.130 安装驱动和配置

4.6.3.131 安装驱动和配置

4.6.3.132 安装驱动和配置

4.6.3.133 安装驱动和配置

4.6.3.134 安装驱动和配置

4.6.3.135 安装驱动和配置

4.6.3.136 安装驱动和配置

4.6.3.137 安装驱动和配置

4.6.3.138 安装驱动和配置

4.6.3.139 安装驱动和配置

4.6.3.140 安装驱动和配置

4.6.3.141 安装驱动和配置

4.6.3.142 安装驱动和配置

4.6.3.143 安装驱动和配置

4.6.3.144 安装驱动和配置

4.6.3.145 安装驱动和配置

4.6.3.146 安装驱动和配置

4.6.3.147 安装驱动和配置

4.6.3.148 安装驱动和配置

4.6.3.149 安装驱动和配置

4.6.3.150 安装驱动和配置

4.6.3.151 安装驱动和配置

4.6.3.152 安装驱动和配置

4.6.3.153 安装驱动和配置

4.6.3.154 安装驱动和配置

4.6.3.155 安装驱动和配置

4.6.3.156 安装驱动和配置

4.6.3.157 安装驱动和配置

4.6.3.158 安装驱动和配置

4.6.3.159 安装驱动和配置

4.6.3.160 安装驱动和配置

4.6.3.161 安装驱动和配置

4.6.3.162 安装驱动和配置

4.6.3.163 安装驱动和配置

4.6.3.164 安装驱动和配置

4.6.3.165 安装驱动和配置

图 A-6 · 组间比较。6 张第 16 天脑类器官照片按 groups.csv 分成 wt2D 与 TH2-7 两组。助手做 Mann-Whitney 检验与 Cliff's delta, 给出各组中位数, 并指出每组 3 个样本时检验不可能显著、需要至少 4 个。

已完成。PODACI 的验证计划不能直接进行，下面说明原因。

速写计数

图片	模型尺寸大小	驱动已部署	建议采用
PODAC1	217	7	约 42-45
PODAC2	89	4	89
PODAC3	46	11	46

为什么 PODAC1 要变

PODAC1 使用模型时，podac 模型型别将变量大小点也做成了目标，导致直接去为图是困难的(例如)

- 16-32 个 的 170 一个 - 单眼 / 眼睛
- 48-64 个 的 170 一个 - 眼睛
- 48-64 个 的 42 的一个 - 眼睛
- 48-64 个 的 42 的一个 - 眼睛

目前 overlay 也已经 照那些 2D+2D 像素点的平面在数据层上，不兼容需要。YOLO 特征数计数器，同样也是 16-32 个

PODAC1 和 PODAC2 的图像分辨率是不同的，没有这个限制，overlay 这个模块只能部分融合操作，所以只能看可用。(PODAC 的 YOLO 是 30 左右，最大目标是检测他们的检测精度，以分析是否可行。)

两点注意

1. 检测到大量数据时不能 (PODAC2 是高度变化，中心位置 3029 个; PODAC1 是高度变化，中心 382 个)，所以不能直接使用同一个尺寸来统计，也会导致数据上“每个至少多少个”的问题——PODAC1 对 7 个 PODAC2 的 46 个模型的实验结果如下。
2. 有数据量大的时候，也有数据量小的时候。这会影响模型 yolo 包 (它包含模型和数据包) 的输入，所以只能看可用。

文件

- 路径：~/.jupyter/jupyter_notebook_config.py 20808012_20808012_244857_7aa3ca/report.html
- 是个临时目录 同时查看 path/features.csv 查看 overlayLayers
- 计数信息：output/pdac_counts.csv

如果单独确认 PODAC1 只是其具类，我们可以设置 340 个单元导出出一组新的 features。

图 A-9 · 器官未知。3 张胰腺癌类器官照片，用户没说器官。助手按约定用通用模型并声明精度较低，或反问器官类型。

这个目录是空的。需要没有任何文件（连隐藏文件也没有），所以无法分析。

```
//root/autodl-fs/AI45/e5/data/empty/ - 0 个文件
```

需要确认一下：

1. 图片是不是还没上传/传到了别处？ 同级的 /root/autodl-fs/AI45/e5/data/ 下已知有 intestine/, mixed/, groups/, series/ 这几个目录都是有图的，你要分析的是不是其中之一？
2. 如果确实是新的一些数据，告诉我实际路径，我再路分析。

另外，为了出准确结果，开始前我还需要两个信息（和之前几次一样）：

- 器官类型（肠 / 胰 / 其他，决定用哪个分割模型）
- 像素尺寸 `um/px`（没有的话结果只能以像素为单位）

如果只是想知道“有几个”，可以用绘图计数（`count_organoids`）更快；要每个类器官的大小形态仍用 `analyze_images`。

图 A-10 · 空目录。目录里没有图片。助手明确说明没有找到图片，而不是返回一个看起来正常的数字。